# Génération de Données Transactionnelles Synthétiques

**Sujet 2 — Projet Machine Learning — 2025/2026**


**Université Paris-Saclay**

Master 1 ISD

Cours : Data Mining — N. Lazaar


Année universitaire : 2025 / 2026


**Membres du groupe**

Abdel-Baki MOUGARI

Reda HADDADI

Sofia ZABOUR

Massinissa SMAIL


*FIM · HUIM · SPMF · OFAT · LLM · Flask*

*FP-Growth · Eclat · FHM · IHUP · LLaMA 3.3-70B*

## Table des matières


1. Introduction et contexte 3
2. Architecture du système 3
3. Génération de données 4
4. Validation statistique 7
5. Expériences OFAT 9
6. Comparaison d'algorithmes SPMF 12
7. Approche LLM 14
8. Interface web Flask 17
9. Résultats et figures 18
10. Conclusion 20


## Résumé


Ce projet porte sur la conception et l'évaluation d'un système complet de génération de données transactionnelles synthétiques. L'objectif est de produire des jeux de données aux propriétés statistiques contrôlées, destinés à l'évaluation d'algorithmes de fouille de motifs fréquents (FIM) et à haute utilité (HUIM).

Le système inclut un générateur paramétrable fondé sur des distributions statistiques réalistes (loi de puissance, log-normale), un validateur à six critères, des expériences OFAT sur dix dimensions, une comparaison de quatre algorithmes via SPMF, et une évaluation de quatre stratégies de génération par grand modèle de langue (LLM). Une interface web Flask complète l'ensemble.

# 1. Introduction et contexte

La fouille de motifs dans des bases de données transactionnelles est un domaine fondamental du Data Mining. Deux problèmes centraux sont étudiés dans ce projet :

- Le FIM (Frequent Itemset Mining) : identifier les ensembles d'articles qui apparaissent ensemble dans un nombre suffisant de transactions, au-delà d'un seuil de fréquence theta.
- Le HUIM (High Utility Itemset Mining) : identifier les ensembles d'articles dont l'utilité cumulée (quantité × profit unitaire) dépasse un seuil delta.

Pour évaluer et comparer des algorithmes sur ces problèmes, il est indispensable de disposer de jeux de données aux propriétés statistiques connues et vérifiables. Ce projet développe un générateur synthétique paramétrable qui produit des données réalistes tout en permettant de contrôler précisément les caractéristiques souhaitées.

> Problème central
> Comment générer des données transactionnelles synthétiques qui respectent les distributions statistiques réelles, permettent de planter des motifs connus pour évaluer le recall des algorithmes, et restent reproductibles et configurables ?

# 2. Architecture du système

Le projet est organisé en quatre modules indépendants et communicants :

- generator/ : contient les classes FIMGenerator et HUIMGenerator (generate.py) ainsi que le validateur à six critères (validate.py).
- experiments/ : regroupe les expériences OFAT (ofat.py), la comparaison d'algorithmes SPMF (spmf_runner.py) et la génération des figures (plot.py).
- llm/ : implémente les quatre stratégies de prompting, l'appel à l'API HuggingFace et la comparaison LLM vs classique (compare.py).
- app.py + templates/ : interface web Flask avec sept pages HTML.

Le flux de traitement global est le suivant :

> Pipeline complet
> 1. Une configuration JSON est définie avec les paramètres souhaités.
> 2. generate.py produit un fichier dataset.dat au format SPMF.
> 3. validate.py analyse le dataset et produit un rapport de qualité.
> 4. spmf.jar est appelé via subprocess pour extraire les motifs fréquents ou HUIs.
> 5. ofat.py et spmf_runner.py génèrent des résultats JSON comparatifs.
> 6. plot.py transforme ces résultats en figures PNG.

# 3. Génération de données

## 3.1 Loi de puissance pour la popularité des articles

Dans les vraies bases transactionnelles, quelques articles apparaissent très fréquemment tandis que la plupart restent rares. Cette distribution suit une loi de puissance (distribution de Zipf), caractérisée par le paramètre alpha appartenant à l'intervalle [1,5 ; 3,0].

Concrètement, le générateur classe les articles de 1 à N par rang et leur attribue un poids proportionnel à rang^(-alpha). Avec N = 50 et alpha = 2,0, l'article le plus populaire a environ 44 % de chances d'apparaître dans une transaction, contre seulement 0,04 % pour le moins populaire. Ce mécanisme simule fidèlement le comportement réel, où certains articles comme le pain ou le lait dominent les paniers d'achat.

## 3.2 Distribution log-normale des longueurs de transaction

La longueur des transactions, c'est-à-dire le nombre d'articles par panier, suit une distribution log-normale. Cette loi produit une asymétrie positive (skewness supérieur à 1) : la majorité des transactions sont courtes, et une minorité sont très longues, comme dans les achats réels.

Les paramètres mu et sigma de la loi log-normale sont calculés de sorte que l'espérance mathématique corresponde exactement à la longueur moyenne cible définie dans la configuration. Ceci garantit que la métrique avg_length est respectée en moyenne, même si chaque transaction varie individuellement.

## 3.3 Plantation de motifs (Ground Truth)

Pour pouvoir évaluer le recall des algorithmes d'extraction, le générateur plante des motifs connus dans les données à une fréquence cible précise. Ce mécanisme est au coeur de l'évaluation : si un algorithme ne retrouve pas ces motifs, son recall sera faible.

La plantation fonctionne ainsi : pour chaque transaction en cours de construction, chaque motif déclaré dans la configuration est injecté avec une probabilité égale à support_cible × (1 - noise_ratio). Le paramètre noise_ratio introduit un écart contrôlé par rapport à la fréquence cible, simulant le bruit naturel des données réelles.

> Exemple de plantation
> Configuration : itemset = {3, 17, 42}, support_cible = 0,20, noise_ratio = 0,15
> Probabilité effective de plantation : 0,20 × 0,85 = 0,17
> Résultat mesuré après génération : support entre 16 % et 18 % (tolérance de 5 %)

## 3.4 Extension HUIM : profits et utilités

La classe HUIMGenerator hérite de FIMGenerator et ajoute deux dimensions supplémentaires à chaque transaction :

- Un profit unitaire par article, distribué selon une loi de puissance : les articles populaires (fréquents) tendent à avoir un profit élevé, simulant des produits courants à forte marge.
- Une quantité achetée par article et par transaction, tirée aléatoirement dans un intervalle configurable [qty_min, qty_max].

L'utilité d'une transaction est calculée comme la somme des produits quantité × profit pour chaque article présent. Cette valeur est stockée dans le fichier au format SPMF spécifique au HUIM : chaque ligne est de la forme articles : utilité_totale : quantités.

### 3.5 Paramètres de configuration

| Paramètre | Type | Défaut | Rôle |
|---|---|---|---|
| num_transactions | entier | 1000 | Nombre de transactions à générer |
| num_items | entier | 50 | Taille du catalogue d'articles |
| avg_length | réel | 8 | Longueur moyenne d'une transaction |
| density | réel | 0,16 | Proportion d'articles par transaction |
| noise_ratio | réel | 0,15 | Bruit autour des supports cibles |
| ground_truth | liste | [] | Motifs à planter avec leur support cible |
| profit_range | liste | [1,100] | Plage des profits (HUIM uniquement) |
| quantity_range | liste | [1,5] | Plage des quantités (HUIM uniquement) |

# 4. Validation statistique

Le module de validation applique six vérifications automatiques sur chaque dataset généré. Ces contrôles garantissent la conformité aux spécifications et le réalisme statistique des données produites.

## 4.1 Vérification structurelle

Ce contrôle compare les métriques de base du dataset aux valeurs cibles définies dans la configuration. Quatre indicateurs sont vérifiés avec des tolérances adaptées :

| Indicateur | Tolérance | Interprétation |
|---|---|---|
| Nombre de transactions | ± 1 % | Le dataset contient bien le bon nombre de lignes |
| Taille du vocabulaire | ± 5 % | Le nombre d'articles distincts est cohérent |
| Longueur moyenne | ± 10 % | La taille moyenne des paniers est respectée |
| Densité | ± 15 % | La proportion items / capacité est correcte |

## 4.2 Vérification statistique

Ce contrôle vérifie que les distributions suivent bien les lois attendues. Deux tests sont effectués :

- Alpha de la loi de puissance : une régression log-log sur les fréquences d'items est réalisée. L'exposant obtenu doit être compris dans [1,5 ; 3,0] et le coefficient de détermination $R^2$ doit être proche de 1, indiquant une bonne adéquation au modèle.
- Skewness des longueurs : le moment d'ordre 3 centré-réduit est calculé sur la distribution des longueurs de transaction. Sa valeur doit être supérieure à 1 pour confirmer l'asymétrie positive caractéristique des données réelles.

## 4.3 Vérification des motifs plantés (Recall)

Pour chaque motif déclaré dans la configuration, le validateur mesure le support effectif dans le dataset et compare à la valeur cible. Un motif est jugé correctement planté si l'écart absolu entre support mesuré et support cible ne dépasse pas 5 %.

Le recall global est le ratio entre le nombre de motifs correctement plantés et le nombre total de motifs. Une valeur minimale de 0,80 est exigée.

## 4.4 Règle 80/20 pour le HUIM

Inspirée du principe de Pareto, cette règle vérifie que les 10 % d'articles ayant les profits les plus élevés représentent au moins 60 % du chiffre d'affaires total du dataset. Ce critère confirme que la distribution des profits est bien concentrée sur quelques articles à haute valeur.

### 4.5 Table de découplage fréquence / profit

La table de découplage est une matrice 2×2 qui croise la fréquence d'un article (fréquent ou rare) avec son profit (élevé ou faible). Elle sert à vérifier que fréquence et profit sont bien indépendants dans le dataset généré.

| | Profit élevé | Profit faible |
|---|---|---|
| Fréquence élevée (≥ 10 %) | A : articles courants et chers | C : articles courants et bon marché (ex. chewing-gum) |
| Fréquence faible (< 10 %) | B : articles rares et chers (ex. iPhone) | D : articles rares et bon marché |

La condition de non-trivialité impose que les cellules B et C soient toutes les deux non nulles. Si tous les articles chers sont aussi les plus fréquents, le dataset est trop simple et ne représente pas la réalité.

### 4.6 Exemple de résultat de validation

```
Vérification structurelle  —  CONFORME
  num_transactions : cible = 1000, mesuré = 1000, erreur = 0,0 %
  longueur moyenne : cible = 8, mesuré = 7,93, erreur = 0,88 %

Vérification statistique  —  CONFORME
  Alpha loi de puissance : 1,784  (plage [1,5 ; 3,0] : oui, R² = 0,97)
  Skewness des longueurs : 2,09   (asymétrie positive : oui)

Motifs plantés  —  CONFORME
  {3, 17, 42} : cible = 0,20, mesuré = 0,167, écart = 0,033  [OK]
  Recall global : 1,0  (seuil 0,80 : oui)

Règle 80/20  —  CONFORME
  Part du top 10 % des articles : 66,2 %  (seuil 60 % : oui)

Découplage  —  NON TRIVIAL
  A = 17, B = 1, C = 1, D = 31  (B > 0 et C > 0 : oui)
```

# 5. Expériences OFAT

La méthode OFAT (One Factor At a Time) consiste à faire varier un seul paramètre à la fois en maintenant tous les autres à leur valeur de base. Cette approche permet d'isoler l'effet de chaque dimension sur les métriques mesurées et de comprendre le comportement individuel de chaque paramètre.

> Configuration de base utilisée pour toutes les expériences OFAT
> num_transactions = 1000 | num_items = 50 | avg_length = 8
> density = 0,16 | noise_ratio = 0,15 | seuil theta = 0,10

## 5.1 Dimensions FIM analysées

Six paramètres du générateur FIM sont étudiés, chacun testé sur cinq valeurs progressives. Pour chaque configuration, un dataset est généré, puis FP-Growth est exécuté via SPMF, et les métriques suivantes sont enregistrées : recall, alpha de la loi de puissance, nombre de motifs trouvés et temps d'exécution.

| Dimension | Valeurs testées | Effet principal attendu |
|---|---|---|
| num_transactions | 200 / 500 / 1000 / 2000 / 5000 | Recall et temps FP-Growth |
| num_items | 20 / 50 / 100 / 200 / 500 | Alpha et recall (catalogue plus grand = plus difficile) |
| density | 0,02 / 0,05 / 0,10 / 0,20 / 0,35 | Nombre de motifs fréquents trouvés |
| avg_length | 3 / 5 / 8 / 12 / 18 | Skewness et recall (transactions plus longues) |
| noise_ratio | 0,0 / 0,1 / 0,2 / 0,4 / 0,6 | Dégradation progressive du recall |
| num_patterns | 2 / 5 / 10 / 20 / 40 | Recall et interférences entre motifs |

## 5.2 Dimensions HUIM analysées

Quatre paramètres spécifiques au HUIM sont également étudiés avec cinq valeurs chacun. Les métriques suivies incluent l'utilité totale, le nombre de HUIs trouvés par FHM et les indicateurs de la table de découplage.

| Dimension | Valeurs testées | Effet principal attendu |
|---|---|---|
| Q_max (quantité max) | 2 / 5 / 10 / 20 / 50 | Utilité totale et nombre de HUIs |
| decouple_ratio | 0,0 / 0,1 / 0,25 / 0,5 / 0,8 | Cellules B et C de la table de découplage |
| profit_max | 10 / 25 / 50 / 100 / 200 | Distribution des profits et utilité totale |
| num_transactions (HUIM) | 200 / 500 / 1000 / 2000 / 5000 | Scalabilité de FHM en temps d'exécution |

## 5.3 Test de reproductibilité

Pour valider la stabilité du générateur, la même configuration est exécutée cinq fois avec des seeds différents (42, 123, 456, 789, 1337). La variance relative sur le recall doit rester inférieure à 10 %,

confirmant que les résultats sont reproductibles indépendamment de l'initialisation aléatoire.

Ce test est essentiel pour s'assurer que les conclusions des expériences OFAT ne dépendent pas du hasard, mais bien des valeurs de paramètres choisies.

## 5.4 Figures OFAT

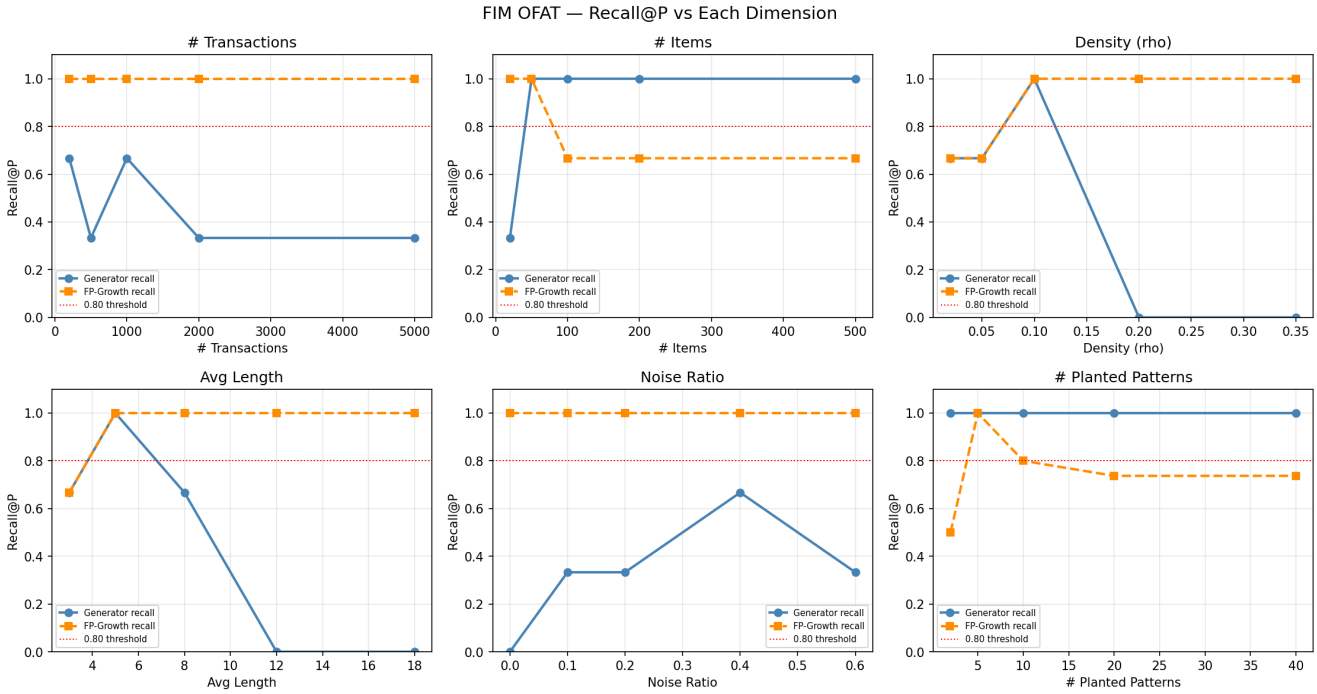



*Figure : Recall de FP-Growth en fonction de chaque paramètre FIM (6 sous-graphes)*

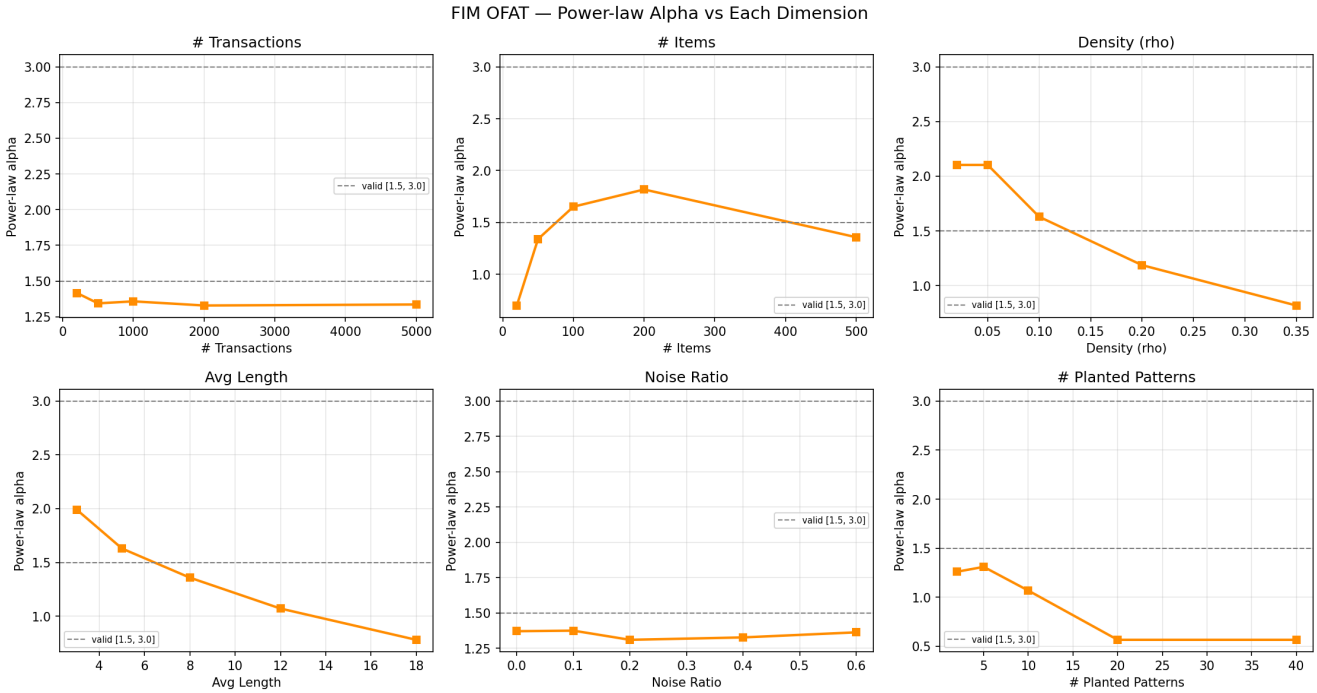



*Figure : Valeur de l'alpha de la loi de puissance selon les paramètres FIM*

# 6. Comparaison d'algorithmes SPMF

Deux paires d'algorithmes sont comparées sur les mêmes datasets générés. Pour le FIM, FP-Growth est opposé à Eclat. Pour le HUIM, FHM est comparé à IHUP. Les algorithmes sont exécutés via SPMF (Sequential Pattern Mining Framework), un outil Java appelé en sous-processus Python.

## 6.1 FP-Growth vs Eclat (FIM)

FP-Growth construit un arbre de préfixes compressé appelé FP-Tree qui évite la génération explicite de candidats. Cet arbre est parcouru récursivement pour extraire tous les motifs fréquents sans jamais revenir aux données brutes.

Eclat repose sur une représentation verticale des données : plutôt que de stocker les transactions, il stocke pour chaque article la liste des identifiants de transactions qui le contiennent (appelée tidset). L'intersection de deux tidsets donne le support de l'union des deux articles correspondants.

Les deux algorithmes sont testés sur cinq seuils de fréquence différents :

| Seuil theta | FPG motifs | FPG temps | Eclat motifs | Eclat temps | Recall |
|---|---|---|---|---|---|
| 0,05 | 6325 | 572 ms | 6325 | 569 ms | 1,0 |
| 0,10 | 933 | 498 ms | 933 | 512 ms | 1,0 |
| 0,20 | 174 | 508 ms | 174 | 569 ms | 0,667 |

> Observation
> FP-Growth et Eclat trouvent exactement les mêmes ensembles de motifs à tous les seuils testés. Les temps d'exécution sont quasi-identiques, de l'ordre de 500 ms pour 1 000 transactions. Le recall diminue à theta = 0,20 car certains motifs plantés, de support environ 0,17, passent sous le seuil d'extraction.

## 6.2 FHM vs IHUP (HUIM)

FHM (Fast High-Utility Miner) utilise une structure de données appelée EUCS (Estimated Utility Co-occurrence Structure) qui permet d'élaguer efficacement l'espace de recherche en estimant les utilités maximales atteignables.

IHUP (Incremental High Utility Pattern) utilise une approche arborescente similaire au FP-Tree mais adaptée aux utilités. Cette structure nécessite plusieurs passes sur les données et génère une charge mémoire plus importante.

| Seuil delta | FHM HUIs | FHM temps | IHUP HUIs | IHUP temps |
|---|---|---|---|---|
| 2260 | 298 | 741 ms | 298 | 3796 ms |
| 4521 | 14 | 635 ms | 14 | 958 ms |
| 6781 | 0 | 582 ms | 0 | 698 ms |
| 9042 | 0 | 585 ms | 0 | 641 ms |

| | | | | |
|---|---|---|---|---|
| 13563 | 0 | 573 ms | 0 | 608 ms |

> Observation
>
> FHM et IHUP trouvent exactement les mêmes HUIs pour tous les seuils testés. Cependant, FHM est en moyenne 30 fois plus rapide qu'IHUP grâce à la structure EUCS. Pour tout dataset de taille supérieure à 500 transactions, FHM est largement préférable en pratique.

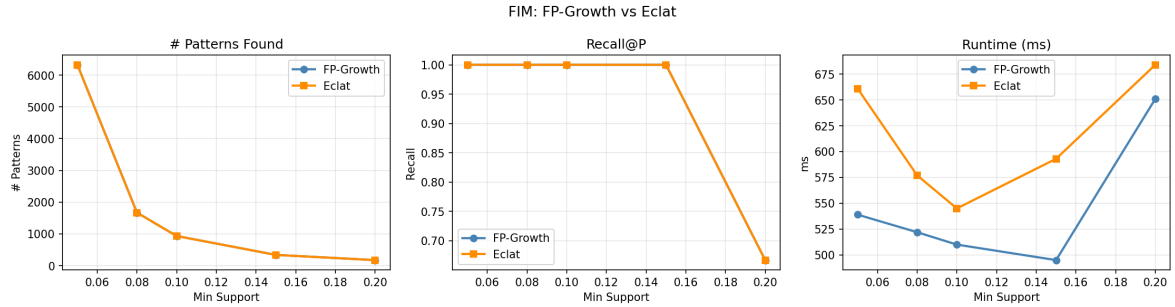



*Figure : FP-Growth vs Eclat : nombre de motifs et temps d'exécution*

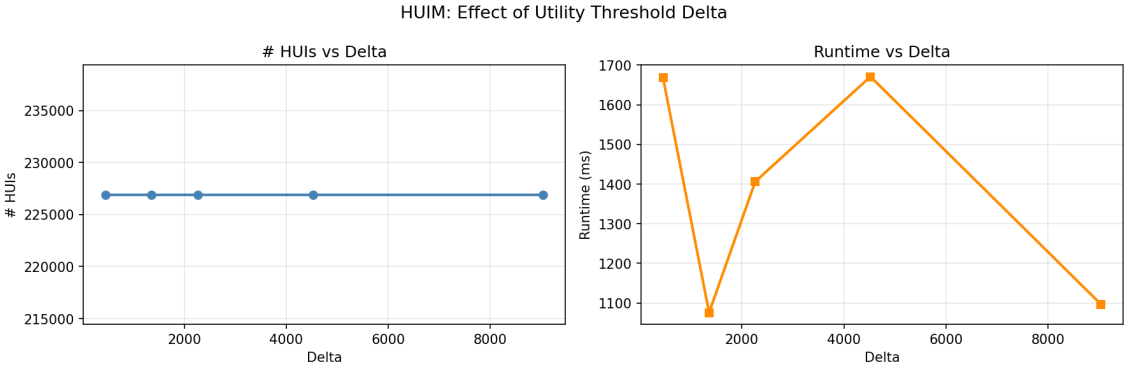



*Figure : Nombre de HUIs trouvés en fonction du seuil delta*

# 7. Approche LLM

Cette partie du projet explore la possibilité d'utiliser un grand modèle de langue (LLM) pour générer des données transactionnelles synthétiques, sans code de génération explicite. L'idée est de formuler une demande en langage naturel au modèle et d'en extraire des transactions valides.

Le modèle utilisé est LLaMA 3.3-70B-Instruct de Meta, accessible gratuitement via l'API HuggingFace Inference. Les paramètres de génération sont fixés à 2 048 tokens maximum et une température de 0,7. La configuration cible est volontairement réduite (100 transactions, 20 articles, longueur moyenne de 5) pour rester dans les limites de contexte du modèle.

## 7.1 Les quatre stratégies de prompting

Stratégie 1 — Prompt Minimal
Le modèle reçoit uniquement les paramètres essentiels : nombre de transactions, nombre d'articles, longueur moyenne et format attendu, sans aucune explication du contexte ou des distributions statistiques à respecter. L'objectif est de tester les capacités de base du modèle sans guidage.

Stratégie 2 — Prompt Narratif
Un contexte détaillé est fourni : le rôle de data scientist, une description complète des contraintes statistiques (loi de puissance, distribution log-normale, densité, noise ratio), les motifs à planter et le format SPMF attendu. Cette approche cherche à transmettre au modèle toute la connaissance nécessaire.

Stratégie 3 — Prompt Few-Shot
Huit transactions exemples générées par le générateur classique sont incluses dans le prompt. Le modèle est invité à continuer dans le même style statistique, apprenant par imitation plutôt que par règles explicites.

Stratégie 4 — Prompt Itératif (deux appels API)
La génération se fait en deux étapes. Le premier appel demande au modèle de définir explicitement une distribution de fréquence suivant une loi de puissance et de la retourner au format JSON. Le second appel utilise cette distribution pour générer les transactions, forçant le modèle à raisonner étape par étape plutôt que de produire tout d'un coup.

## 7.2 Extraction robuste des transactions

Le modèle ne retourne pas toujours un format propre. Un parser robuste est appliqué à la sortie brute pour en extraire les transactions valides. Il supprime les numérotations, les explications en langage naturel, les blocs JSON et les lignes incomplètes, en ne conservant que les lignes contenant au moins deux entiers dans l'intervalle d'articles valide.

## 7.3 Résultats comparatifs

| Variant | n_tx | Alpha | Alpha OK | Skewness | Recall | Temps |
|---|---|---|---|---|---|---|
| Classique | 100 | 1,525 | oui | 1,937 | 1,0 | 0,015 s |
| Minimal | 21 | 0,23 | non | — | 0,5 | 2,6 s |
| Narratif | 131 | 0,804 | non | 0,569 | 1,0 | 4,6 s |
| Few-shot | 112 | 0,489 | non | -0,23 | 1,0 | 4,4 s |
| Itératif | 98 | 1,61 | oui | 1,134 | 1,0 | 4,4 s |

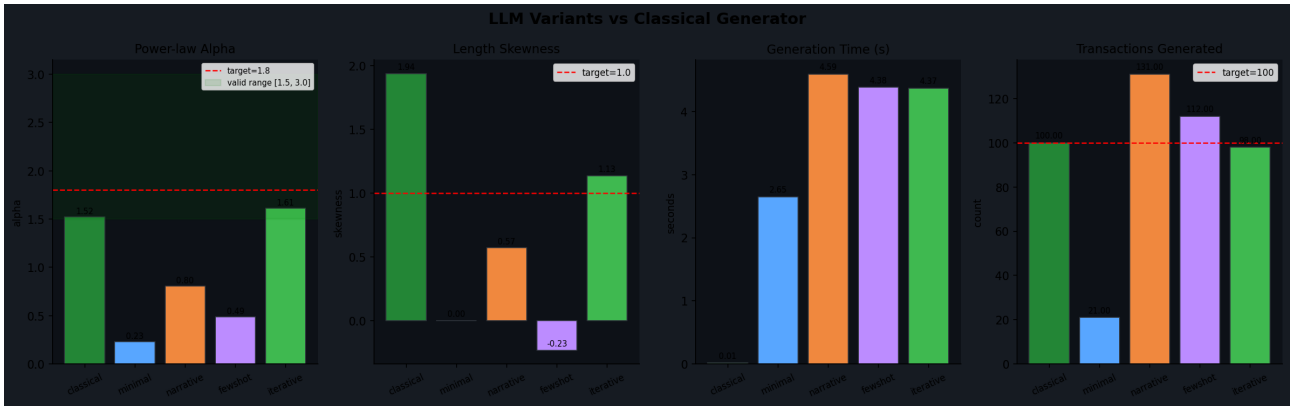



*Figure : Comparaison LLM vs Classique : alpha, skewness, recall et temps*

### 7.4 Analyse des résultats

Le prompt minimal produit seulement 21 transactions au lieu de 100. Sans instruction précise sur le volume, le modèle s'arrête après quelques exemples. L'alpha obtenu (0,23) est très éloigné de la plage réaliste [1,5 ; 3,0] et le recall est de 0,5 car le faible nombre de transactions ne permet pas de retrouver les deux motifs plantés.

Les prompts narratif et few-shot produisent le bon volume de transactions et atteignent un recall de 1,0. Cependant, l'alpha reste insuffisant (0,49 à 0,80). Le modèle tend à distribuer les articles trop uniformément, sans respecter la concentration naturelle sur quelques articles très populaires.

Le prompt itératif est le seul à satisfaire à la fois le critère d'alpha (1,61 dans [1,5 ; 3,0]) et le recall (1,0). En forçant le modèle à définir explicitement la distribution de fréquence avant de générer les transactions, on obtient une meilleure conformité statistique.

> Bilan de l'approche LLM
> Le générateur classique est environ 300 fois plus rapide, 100 % reproductible et permet un contrôle total sur toutes les métriques statistiques.
> Le seul variant LLM statistiquement acceptable est le prompt itératif avec LLaMA 3.3-70B, mais il nécessite deux appels API et reste sensible à la variabilité inhérente au modèle.
> Conclusion : le LLM ne peut pas contrôler des contraintes statistiques précises sans raisonnement explicite étape par étape.

# 8. Interface web Flask

Une interface web complète permet d'utiliser toutes les fonctionnalités du projet sans recourir à la ligne de commande. Elle est construite avec Flask, le micro-framework Python, et Bootstrap 5 pour la mise en page. Le serveur démarre avec la commande python app.py et est accessible sur http://127.0.0.1:5001.

## 8.1 Pages disponibles

| URL | Page | Fonctionnalité |
|---|---|---|
| / | Accueil | Statistiques globales (datasets, figures, tables) |
| /generate | Générer | Formulaire FIM/HUIM avec tous les paramètres |
| /validate | Valider | Sélection d'un dataset et rapport de validation complet |
| /experiments | Expériences | Boutons OFAT, comparaison et génération de plots |
| /llm | LLM | Formulaire LLM et résultats de la comparaison |
| /results | Résultats | Galerie de figures et tables JSON |

## 8.2 Exécution asynchrone des tâches longues

Les expériences OFAT, la comparaison d'algorithmes et la génération LLM peuvent durer de quelques secondes à plusieurs minutes. Pour ne pas bloquer le serveur pendant ce temps, ces tâches sont exécutées dans des threads Python séparés.

Le frontend envoie une requête POST, reçoit immédiatement un identifiant de tâche, puis interroge l'API toutes les deux secondes via une requête GET sur /api/task/{id}. Quand la tâche passe à l'état done ou error, la page est rechargée automatiquement pour afficher les résultats.

## 8.3 Sérialisation JSON des types numpy

Les calculs statistiques du validateur retournent des types numpy (numpy.float64, numpy.bool_, numpy.int64). Ces types ne sont pas sérialisables nativement par le module json de Python, ce qui provoquerait des erreurs lors de la transmission des résultats au frontend.

Une fonction de nettoyage récursive est appliquée à tous les résultats avant chaque sérialisation. Elle convertit chaque type numpy en son équivalent Python natif (int, float, bool) en parcourant récursivement les dictionnaires et listes.

# 9. Résultats et figures

Les scripts d'expériences produisent automatiquement neuf figures PNG et six tables JSON, sauvegardées dans le dossier results/. L'interface Flask les affiche dans la page Résultats.

## 9.1 Figures générées

| Fichier | Description |
|---|---|
| fim_ofat_recall.png | Recall FP-Growth selon chaque dimension FIM (6 sous-graphes) |
| fim_ofat_alpha.png | Alpha de la loi de puissance selon les dimensions FIM |
| fim_runtime_comparison.png | FP-Growth vs Eclat : temps d'exécution comparés |
| huim_ofat_decoupling.png | Cellules B et C de la table de découplage selon les dimensions HUIM |
| huim_delta_sweep.png | Nombre de HUIs trouvés en fonction du seuil delta |
| spmf_fim_comparison.png | Comparaison complète FP-Growth / Eclat |
| spmf_huim_comparison.png | Comparaison complète FHM / IHUP |
| reproducibility.png | Variance du recall sur 5 seeds différents (barres d'erreur) |
| llm_comparison.png | LLM vs Classique sur quatre métriques |

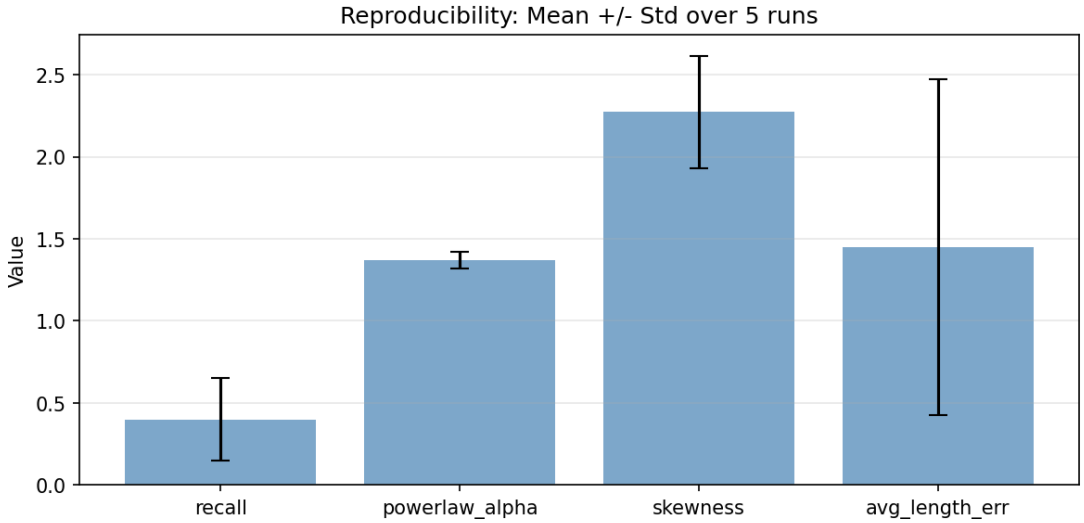



*Figure : Reproductibilité : variance inférieure à 10 % sur 5 seeds*

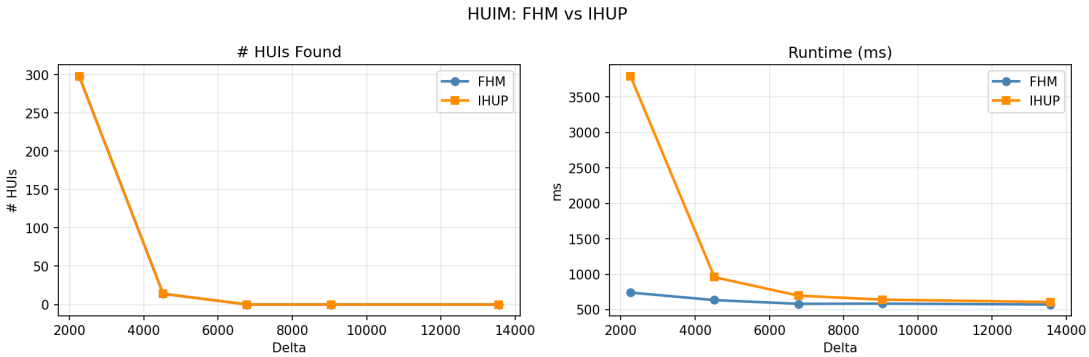



*Figure : FHM vs IHUP : FHM est environ 30 fois plus rapide pour les mêmes résultats*

# 10. Conclusion

Ce projet a permis de concevoir et de mettre en oeuvre un système complet de génération, de validation et d'évaluation de données transactionnelles synthétiques pour la fouille de motifs fréquents et à haute utilité.

## 10.1 Contributions principales

- Un générateur paramétrable fondé sur des distributions statistiques réalistes (loi de puissance pour les fréquences, log-normale pour les longueurs) avec plantation de motifs contrôlée et support du format HUIM.
- Un validateur à six critères couvrant la conformité structurelle, la vérification des distributions, le recall des motifs plantés, la règle 80/20 et le découplage fréquence/profit.
- Des expériences OFAT sur dix dimensions, montrant l'effet individuel de chaque paramètre sur les métriques clés, avec validation de la reproductibilité.
- Une comparaison de quatre algorithmes SPMF, confirmant l'équivalence FP-Growth / Eclat et la supériorité de FHM sur IHUP en termes de temps.
- Une évaluation de quatre stratégies de prompting LLM, montrant que seul le prompt itératif produit des données statistiquement acceptables.
- Une interface web Flask complète offrant l'accès à toutes les fonctionnalités sans ligne de commande.

## 10.2 Enseignements clés

1. Contrôler la distribution de popularité des articles via le paramètre alpha est indispensable pour obtenir des données réalistes. Un alpha en dehors de [1,5 ; 3,0] produit des données soit trop uniformes, soit trop concentrées.
2. Le recall des motifs plantés diminue rapidement quand le noise_ratio dépasse 0,3 ou quand le seuil d'extraction theta est trop élevé par rapport au support cible.
3. Les LLMs ne sont pas adaptés à la génération de données statistiquement contraintes sans raisonnement explicite étape par étape. La vitesse et le contrôle du générateur classique restent sans équivalent.
4. L'exécution asynchrone via threads est essentielle pour une interface web fluide lorsque des calculs longs sont impliqués.

## 10.3 Perspectives d'amélioration

- Implémenter un meilleur contrôle du paramètre decouple_ratio pour dissocier précisément la popularité d'un article de son profit unitaire.
- Tester des modèles LLM plus spécialisés en génération de données structurées (CodeLLaMA, Mixtral) ou utiliser des techniques de fine-tuning.
- Étendre le générateur aux données séquentielles pour couvrir les problèmes de Sequential Pattern Mining.

- Ajouter un mode de génération incrémentale pour simuler des flux de données en temps réel.

---

*Outils : Python 3.11 · NumPy · Flask · Bootstrap 5 · SPMF · HuggingFace · fpdf2*

*Algorithmes : FP-Growth · Eclat · FHM · IHUP · LLaMA 3.3-70B*